

Dating App Match Prediction - Project Overview & Task Assignment

1. Models, Accuracies, and Logic

The project currently trains **6 Machine Learning models**, alongside 1 `DummyClassifier` (a baseline model that simply guesses the most frequent class to establish a minimum threshold).

Currently, the models are evaluated under two conditions: using `class_weight='balanced'` and using SMOTE. **Note:** The Dummy Baseline achieves **39.70%**, and most of the models are currently hovering around or below this mark, which means they are struggling to find strong predictive signals in the current data.

Here is the breakdown of the 6 models:

1. Logistic Regression

- **Logic:** A linear statistical model that applies a mathematical equation to the features to predict the log-odds of a class. It assumes a linear relationship between features and the target.
- **Accuracy:** 33.94% (Balanced) / 33.27% (SMOTE)

2. Decision Tree

- **Logic:** A rule-based model that recursively splits the dataset into branches based on feature thresholds (e.g., "If Age > 25, go left...").
- **Accuracy:** 33.78% (Balanced) / 34.95% (SMOTE)

3. Random Forest

- **Logic:** An ensemble "bagging" method. It builds hundreds of different decision trees on random subsets of the data and averages their predictions to prevent the overfitting that plagues single Decision Trees.
- **Accuracy:** 33.96% (Balanced) / 37.35% (SMOTE)

4. XGBoost (Extreme Gradient Boosting)

- **Logic:** An advanced ensemble "boosting" method. It builds trees sequentially, where every new tree specifically focuses on correcting the errors made by the previous trees.
- **Accuracy:** 36.56% (Balanced) / 36.79% (SMOTE)

5. SVM (Support Vector Machine / LinearSVC)

- **Logic:** A margin-based geometric model. It plots data points in space and attempts to draw a hyperplane (decision boundary) that maximally separates the different classes with the widest possible gap.
- **Accuracy:** 39.19% (Balanced) / 33.36% (SMOTE)

6. Neural Network (MLPClassifier)

- **Logic:** A Multi-Layer Perceptron. It passes data through hidden layers of artificial "neurons" with non-linear activation functions, allowing it to learn highly complex, non-linear patterns.
 - **Accuracy:** 39.39% (Balanced) / 33.11% (SMOTE)
-

2. Role of Files and Folders

Your project is structured modularly. Here is what each part does:

Folders:

- `data/`: Stores the raw CSV dataset (`dating_app_behavior_dataset.csv`).
- `models/`: Stores the trained, serialized models (`.pkl` files) so you can load them later without having to retrain them from scratch.
- `reports/`: Stores tabular evaluation metrics, like the CSV files comparing model accuracies.
- `visualizations/`: Stores generated plots (EDA graphs, confusion matrices, ROC curves, and SHAP diagrams).
- `notebooks/` & `presentation/`: Workspaces for Jupyter notebooks and presentation assets.
- `src/`: The core source code directory containing reusable modules.

Files:

- `main.py`: The orchestrator script. It ties everything together by calling functions from `src/` to execute the pipeline from start to finish.
 - `requirements.txt`: Lists the external Python libraries (like scikit-learn, xgboost, imblearn) needed to run the code.
 - `src/__init__.py`: Tells Python to treat the `src` folder as an importable module.
 - `src/preprocessing.py`: Handles data loading, cleaning, scaling numerical columns, encoding categorical columns, and splitting data into train/test sets.
 - `src/feature_engineering.py`: Houses logic to combine or alter existing columns to create brand new features.
 - `src/train_models.py`: Defines the 6 models, sets up their hyperparameter tuning grids (`GridSearchCV`), and trains them using pipelines.
 - `src/evaluation.py`: Calculates mathematical metrics (Accuracy, F1, Precision) and builds the comparison tables.
 - `src/visualization.py`: Contains functions to generate all the charts, graphs, and visual explainers.
-

3. What is SMOTE for each model?

SMOTE stands for *Synthetic Minority Over-sampling Technique*.

In your dataset, the target classes are imbalanced (some dating behaviors appear much more frequently than others). If a model is trained on this, it tends to ignore the minority classes and just guess the majority class to look highly accurate.

Instead of just duplicating existing rows to balance the data, **SMOTE mathematically synthesizes entirely new, artificial data points** for the minority classes by interpolating

between existing ones.

- **For each model:** In `train_models.py`, SMOTE is injected into a pipeline *right before* the model is trained. This forces the model to learn from a perfectly balanced dataset, preventing bias. The code then compares the SMOTE approach against the `class_weight='balanced'` approach to see which yields better predictive power.
-

4. Task Assignment Plan (6 Members)

Because the models are currently performing around the Dummy Baseline (~39%), the data is likely very noisy. To push past this, you need both model-specific tuning and global pipeline upgrades.

To ensure fairness, **each member will take ownership of ONE specific model** (to research and tune it) **AND ONE global pipeline improvement** (which benefits the entire project).

Member	Assigned Model	Global Pipeline Task	Description of Global Task
Member 1	Logistic Regression	Feature Engineering Optimization	Linear models need strong, clear signals. This member will analyze the dataset to create new, highly correlated features and handle severe outliers.
Member 2	Decision Tree	Feature Selection / Dimensionality Reduction	Trees can easily overfit on noisy data. This member will implement techniques like PCA or <code>SelectKBest</code> to drop irrelevant columns and keep only the strongest predictors.
Member 3	Random Forest	Advanced Resampling Techniques	Since standard SMOTE isn't dominating, this member will explore alternative balancing techniques like SMOTEENN, ADASYN, or Undersampling to see what helps the models learn best.
Member 4	XGBoost	Advanced Hyperparameter Tuning Setup	This member will upgrade <code>train_models.py</code> to use <code>RandomizedSearchCV</code> or <code>Optuna</code> . This will allow the whole team to find better parameters for their models much faster than the current <code>GridSearchCV</code> .
Member 5	SVM (Support Vector)	Error Analysis & Target Re-evaluation	This member will analyze the confusion matrices to figure out <i>why</i> models are failing (e.g., are two target classes practically identical?). They will suggest if certain target classes should be merged.
Member 6	Neural Network (MLP)	Model Ensembling (Stacking/Voting)	This member will write the code to build a final <code>VotingClassifier</code> or <code>StackingClassifier</code> that aggregates the predictions of Members 1-5 to create one ultimate, high-performing model.

Recommended Workflow:

1. **Phase 1 (Data Fixes):** Members 1, 2, and 3 push their pipeline improvements (Feature

Engineering, Selection, and Resampling).

2. **Phase 2 (Model Tuning):** With the newly improved dataset, all 6 members focus purely on tuning their assigned models, utilizing Member 4's Advanced Hyperparameter Tuning script.
3. **Phase 3 (Final Polish):** Member 5 analyzes the new errors to squeeze out extra percentage points, and Member 6 combines everyone's best models into the final Ensemble.